

MCP-Funktionsreferenz & Workflows

Funktionsbeschreibung der drei MCP-Server (ai-rem, mykeyvault, tools-mcp) und der typischen End-to-End-Abläufe. Topologie siehe `architecture.md`.

Die Tool-Listen beschreiben den **aktuell registrierten** Stand (was Claude in der Session tatsächlich aufrufen kann). Wo der Repo-/README-Stand zusätzliche Helfer definiert, ist das vermerkt.

1. ai-rem — Langzeit-Gedächtnis (Knowledge Graph)

Persistenter Kontext als Graph aus **Entities** (typisiert) und **Relations**. Einzige Quelle für dauerhaftes Wissen. Befüllt wird er **proaktiv** (`memory_add`/`memory_relate` während der Arbeit, Workflow B) und über den **ai-rem Ingest-Hook** am Session-Ende (Workflow C). Nicht zu verwechseln mit Claude Codes **nativem** datei-basiertem Auto-Memory (Setting `autoMemoryEnabled`) — das ist bewusst **deaktiviert**, damit ai-rem die einzige Memory-Senke bleibt.

Entity-Typen: `Person` · `Project` · `Task` · `Tool` · `Problem` · `Solution` · `Decision` · `Preference` · `Topic`

Funktionen

Tool	Zweck
<code>memory_get_context</code>	Relevanten Subgraph laden. Ohne Topic: offene Tasks + aktive Projekte + letzte Einträge + gepinnte Preferences. Mit Topic: direkt passender Ausschnitt. Session-Start-Tool .
<code>memory_search</code>	Hybride Suche (lexikalisch + semantisch) über Name/Beschreibung; kürzt lange Bodies (~120 Zeichen).
<code>memory_search_full</code>	Wie <code>search</code> , aber mit vollständiger Beschreibung (keine Kürzung).
<code>memory_add</code>	Entity anlegen oder aktualisieren (Upsert nach Name). Felder: name, type, description, context, pinned, extra (JSON).
<code>memory_relate</code>	Gerichtete Beziehung zwischen zwei Entities anlegen (z. B. NUTZT, ARBEITET_AN, GELÖST_DURCH, HÄNGT_AB_VON).
<code>memory_get_relations</code>	Alle Beziehungen einer Entity anzeigen.

Tool	Zweck
<code>memory_list</code>	Entities nach Typ auflisten.
<code>memo-ry_preference_update</code>	Einzelne Felder einer Preference ändern (context, pinned, sort_order) ohne die Beschreibung zu überschreiben.
<code>memory_merge</code>	Zwei Duplikate nicht-destruktiv zusammenführen.
<code>memory_archive</code>	Entity archivieren statt löschen (Historie bleibt erhalten).
<code>memory_delete</code>	Entity + alle Kanten löschen.
<code>memory_status</code>	Kurzstatus: Anzahl Entities und Relationen.
<code>memory_check_update</code>	Installierte Version vs. Docker Hub prüfen.

Workflows

A) Session-Start → Kontext laden 1. SessionStart-Hook (`system-check.py`) ruft `memory_get_context` auf. 2. Liefert offene Tasks/Pläne, aktive Projekte, letzte Einträge, gepinnte Preferences. 3. Claude arbeitet mit diesem Kontext; Pfade/Funktionsnamen aus Memory vor Empfehlung gegen den Code verifizieren (Memory = Behauptung über damals).

B) Proaktiv speichern (während der Arbeit) 1. Etwas Überraschendes/Nicht-Offensichtliches gelernt → erst `memory_search` (Dublette?). 2. Existiert die Entity: `memory_add` updaten; sonst neu anlegen. 3. Mit `memory_relate` an verwandte Entities hängen (Project↔Task, Problem↔Solution ...).

C) ai-rem Ingest-Hook (Session-Ende) — *nicht* Claude Codes natives Auto-Memory (das ist aus) 1. `auto-memory.py` (PreCompact/SessionEnd) reicht das Transcript an `ai-rem ingest`. 2. Der Extractor schickt es an **Ollama** (`myubuntu:11434` , geladenes Chat-Modell, Default `mistral-small13.2:24b` , `format=json`). 3. Strukturierte Entities/Relations → Bulk-Upsert über `/mcp` . 4. Ollama langsam/kalt/nicht erreichbar → Transcript wandert in die Queue (`pending.jsonl`); `ai-rem catchup` (läuft zu Beginn jedes Hook-Laufs) zieht es nach, sobald Ollama warm/erreichbar ist. 5. Voraussetzung in der Hook-Umgebung: `AI_REM_CLI` (CLI-Pfad) und `AI_REM_ENDPOINT` (z. B. `https://airem.lan/mcp`) — sonst kein Ingest.

D) Nightly-Cleanup 1. Daemon-Thread im Container sucht Dubletten/veraltete Einträge. 2. Ollama fällt Merge-/Archiv-Urteile; klare Fälle werden angewandt, unklare landen in der Review-Queue der Web-UI (`/cleanup`).

2. mykeyvault — Secrets ohne Leak in den Kontext

mykeyvault-mcp ist das MCP-Frontend vor vault-api (REST) → bw serve → Vaultwarden. Designprinzip: Secrets gelangen **nie als Klartext** in den Chat/Prompt.

Funktionen (aktuell registriert)

Tool	Zweck
<code>vault_list_items</code>	Vault-Einträge auflisten (nur Name + Username, keine Secrets).
<code>vault_create_item</code>	Login-Eintrag anlegen (Passwort als Parameter, wird nicht zurückgegeben).
<code>vault_create_ssh_key</code>	SSH-Key-Eintrag anlegen (Private Key wird aus lokaler Datei gelesen, nie als Parameter übergeben).
<code>vault_get_ssh_public_key</code>	Öffentlichen SSH-Key + Fingerprint holen (kein Private Key).

Repo-/README-Stand definiert zusätzlich *secret-injizierende Helfer* (`vault_write_secret`, `vault_run_with_secret`, `vault_run_with_secret_file`), die ein Secret in eine Temp-Datei / Env-Variable geben und nur den Pfad bzw. das Kommando-Ergebnis zurückliefern. Diese sind in der aktuellen Session nicht registriert — bei Bedarf prüfen, ob der Container sie ausspielt.

vault-api REST (hinter dem MCP): `/health`, `/items`, `/item/{name}`, `/secret/{name}`, `/ssh-key/{name}`, `POST /items`, `POST /ssh-keys`, `PUT /items/{name}`, `DELETE /items/{name}` — alle außer `/health` mit Bearer-Token.

Workflows

A) Secret hinterlegen 1. `vault_create_item` mit Name + Passwort (+ optional Username/Notes). 2. Eintrag landet in Vaultwarden; das Passwort taucht nirgends im Antwort-Payload auf.

B) Token-Verteilung an ai-rem (zentral) 1. Der gemeinsame Bearer-Token liegt als Vault-Item `ai-rem-api-token`. 2. `vault-api` liefert ihn (Bearer-geschützt) an Clients; ai-rem holt ihn pro Session im Hintergrund und schreibt den Header nach `~/.claude.json`. 3. Fällt mykeyvault aus → Clients nutzen den zuletzt gecachten Header (fail-soft).

C) SSH-Key anlegen & nutzen 1. `vault_create_ssh_key` liest den Private Key aus einer lokalen Datei und speichert ihn als Vault-Item (Typ 5). 2.

`vault_get_ssh_public_key` gibt Public Key + Fingerprint zur Weitergabe (z. B. `authorized_keys`) — der Private Key verlässt den Vault nie über das MCP.

3. tools-mcp — Scripts als Tools (lokal, live nachgeladen)

Lokaler stdio-MCP-Server am Mac. Registriert Scripts aus `scripts/<name>/` als Tools und synchronisiert sie alle 5 s von der zentralen `tools-registry` (HTTP :3457) — **neue/geänderte Scripts ohne MCP-Neustart.**

Funktionen (aktuell registriert)

Tool	Zweck
<code>list_scripts</code>	Meta-Tool: alle registrierten Scripts + Manifest-Metadaten (inputs/requires) auflisten.
<code>pipeline_run</code>	Meta-Tool: mehrere Script-Aufrufe verketten, mit Variablen-Interpolation zwischen den Schritten.
<code>ai_rem_token</code>	Liest den ai-rem-Bearer-Token aus <code>~/.claude.json</code> (<code>mcpServers.<server>.headers.Authorization</code>) → <code>{token, authorization}</code> .
<code>settings_sync</code>	Synchronisiert Claude-Code <code>settings.json</code> mit dem Template.
<code>subagent_models</code>	Wertet die Subagent-Modellnutzung aus den <code>subagent-*.jsonl</code> -Transcripts aus.
<code>md_to_pdf</code>	Markdown → PDF (rendert auch Mermaid-Blöcke).
<code>pdf_to_text</code>	Text aus PDF extrahieren.
<code>head_lines</code>	Erste N Zeilen einer Datei lesen.
<code>echo</code>	Echo (Smoke-Test).
<code>magic3_design_install</code>	Installiert den magic3-Design-Skill nach <code>~/.claude/skills/magic3-design/</code> .
<code>dotclaudes_install</code>	Verteilt hauseigene Agents (bulk-worker, scan-worker) + Skills nach <code>~/.claude/</code> .

Script-Konvention: je Script ein Verzeichnis mit `manifest.yaml` (name, description, exec, inputs, requires, optional `ai_rem_entity: tool_<name>`) + ausführbarem `run.sh` / `run.py`. Inputs kommen als `INPUT_<NAME>` + `TOOLS_MCP_INPUTS_JSON`; Outputs schreibt das Script nach `<run_dir>/outputs.json`.

Workflows

A) Script-Distribution / Live-Reload 1. Script wird im Repo unter `scripts/<name>/` ergänzt/geändert; `tools-registry` mountet den Ordner read-only. 2. `tools-mcp` (lokal) pollt `/registry`; ändert sich der SHA256-Versionshash, lädt es geänderte Dateien via `/registry/file` und cached nach `~/.cache/tools-mcp/scripts`. 3. Tool wird registriert/aktualisiert/entfernt; SDK meldet `tools/list_changed` → sofort nutzbar.

B) Script ausführen 1. Claude ruft das Tool mit den Manifest-Inputs auf. 2. `tools-mcp` legt ein Run-Dir (`/tmp/tools-runs/<uuid>`) an, setzt `INPUT_*` / `TOOLS_MCP_*` und führt `exec` via `execFile` aus. 3. `outputs.json` wird geparkt und als Ergebnis zurückgegeben.

C) Pipeline 1. `pipeline_run` verkettet Schritte; Outputs eines Schritts werden per `${var}` in die Inputs des nächsten interpoliert. Gut für „extrahieren → transformieren → rendern“.

D) Dieses Dokument als PDF - `md_to_pdf` auf `docs/mcp-functions.md` bzw. `docs/architecture.md` rendert inkl. Mermaid-Diagramm.

Zusammenspiel der drei (Gesamt-Workflow)

- 1. Session-Start:** `system-check.py` → ai-rem `memory_get_context`; Bearer-Token (aus mykeyvault) ist im `~/.claude.json`-Header; `tools-mcp` hat seine Scripts vom Registry frisch.
- 2. Arbeit:** Claude nutzt ai-rem-Tools für Kontext/Speichern, mykeyvault-Tools für Secrets (ohne Leak), tools-mcp-Scripts für wiederkehrende Aktionen (PDF, settings-sync, Token-Lookup ...).
- 3. Session-Ende:** der ai-rem Ingest-Hook `auto-memory.py` → Ollama-Extraktion → ai-rem-Upsert (bei kaltem/langsamem Ollama via `pending.jsonl` + `catchup` nachgezogen). Nachts räumt ai-rem den Graphen auf (Ollama-Urteile + Review-Queue).